VeloOverlay Toolkit - User Guide

Version 1.9 | May 2026

Welcome to the VeloOverlay Toolkit desktop app. This guide focuses on the installed Electron version and walks through importing activities, editing overlays, and rendering videos offline.

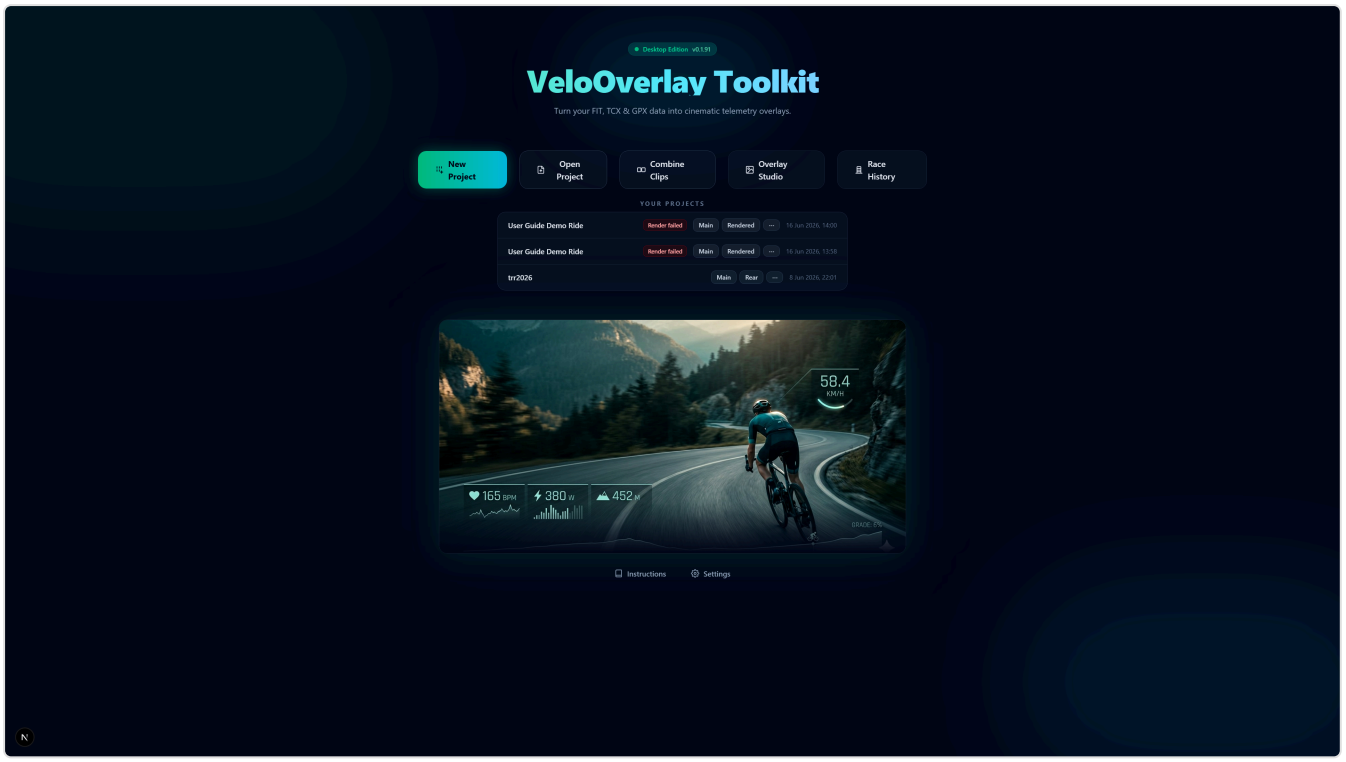
Table of Contents

1. Quick Start (Desktop)
2. Installation
3. First Launch & Settings
4. Updating the App
5. Importing an Activity (FIT/GPX/TCX)
6. Importing from Strava
7. Importing from Garmin Connect
8. Render Notifications (Telegram)
9. Understanding the Editor
10. Timeline Zoom & Scrubbing
11. Multi-Track Trim
12. Media Files Tab
13. Video Playback (Under the Hood)
14. Multi-Clip Video Sequences
15. Canvas Sizes & Square-Source Video
16. GoPro Auto Sync
17. Working with Widgets
18. Icon Picker
19. Comments
20. Output Transform
21. Social Clips
22. Top Moments
23. Flyover Map
24. Estimated Wind
25. HUD Presets & Theme Links
26. Rendering Videos
27. Uploading to YouTube
28. LUT Color Correction
29. Exporting Images
30. Managing Output & Logs

31. Tips & Best Practices
32. Troubleshooting
33. FAQ

Quick Start (Desktop)

1. **Install and launch the app**
    - Windows: run the `.exe` installer.
    - macOS: open the `.dmg` and drag the app into Applications.
    - Linux: run the `.AppImage` (make it executable first).
2. **Open Settings**
    - **Paths tab**: Set **Default Input Folder**, **Default Output Folder**, and **Logs Directory**.
    - **Paths tab**: Choose **FFmpeg Path** if you use a system install.
    - **General tab**: Pick a **Profile Image** for the comments widget (32–200 px).
3. **Import a track**
    - Click **Start new project** and upload a FIT/GPX file.
4. **Design your overlay**
    - Add widgets, position them, and tune styles.
5. **Render**
    - Choose Overlay or Composite mode.
    - When complete, use **Open output folder** to find the result.



Installation

Windows

- Run the installer (unsigned internal build).
- Choose an install location and launch VeloOverlay Toolkit.

macOS

- Open the DMG and drag the app into Applications.
- Launch the app from Applications.

Linux

- Download the AppImage.
- Make it executable and run it:

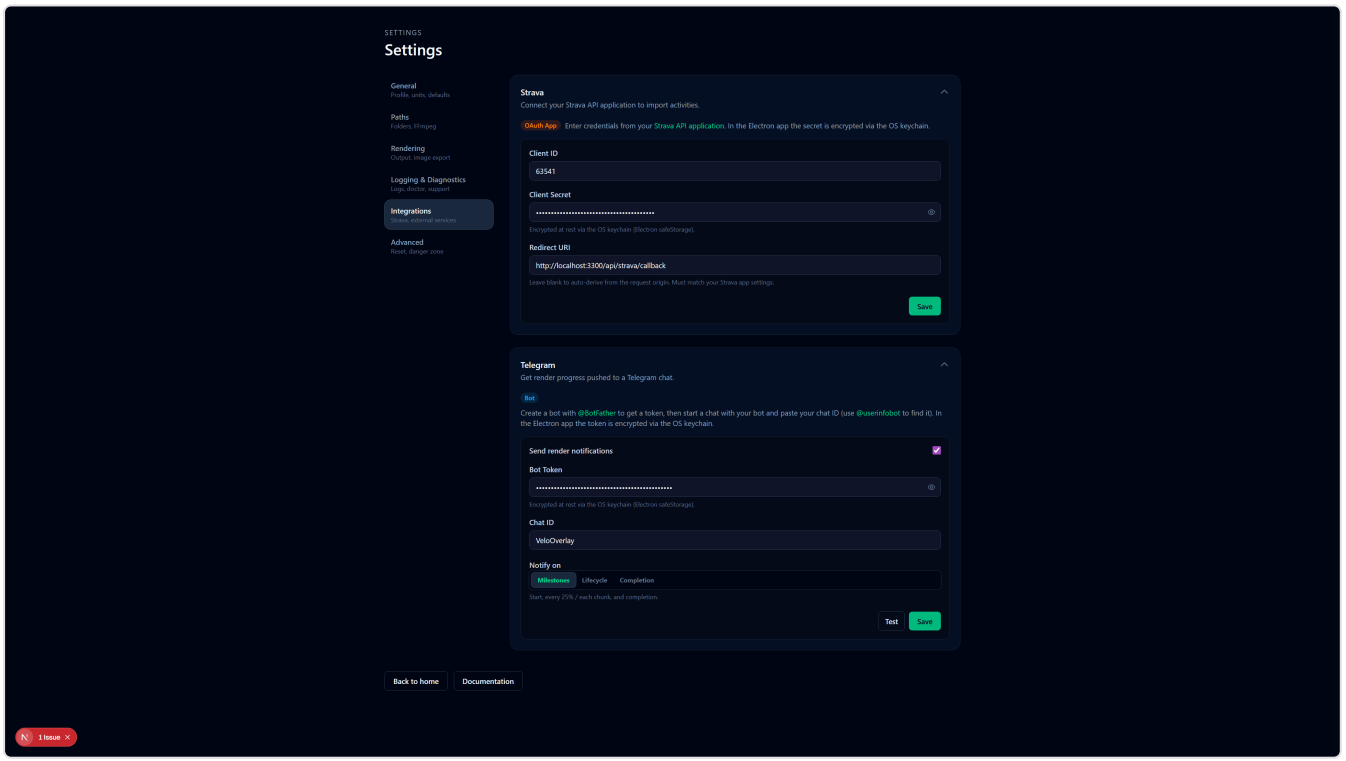
```
chmod +x VeloOverlay.AppImage
./VeloOverlay.AppImage
```

First Launch & Settings

Open **Settings** from the home page. Settings are organized into six tabs in a left sidebar:

Tab	Contents
General	Athlete profile (image, FTP, HR max, cadence ref), units, editor preferences
Paths	Project folders (input/output), logs folder, FFmpeg path
Rendering	Default encoder mode, HEVC preference, image export defaults
Logging & Diagnostics	Log level, usage tracking, doctor check, diagnostic bundle export
Integrations	Strava OAuth credentials; Garmin Connect API key; Telegram notifications
Advanced	Factory reset, danger-zone actions

The active tab is remembered across sessions.



General Tab

- **Profile Image**: Used by the comments widget. Must be 32–200 px (PNG/JPG/WEBP). Click **Choose Image** to pick a file.
- **Athlete Values**: Set your FTP (W), Max Heart Rate (bpm), and Cadence Reference (rpm) for training-zone calculations.
- **Default Units**: Speed (km/h / mph), Distance (km / mi), Elevation (m / ft).
- **Editor → Context Help**: Toggle inline help buttons next to editor controls.
- **About** *(desktop only)*: Shows the installed version and a **Check for updates** button — see Updating the App.

Paths Tab

- **Default Input Folder**: Where you keep source videos (used by video dropdowns).
- **Default Output Folder**: Rendered videos are saved here by default.
- **Logs Directory**: Desktop + web logs are written here.
- **FFmpeg Path**: Optional override if you use a system FFmpeg install.
- **LUT Library Folder** *(desktop only)*: Folder scanned for `.cube` LUT files available in the Video Tuning panel. Defaults to `<userData>/luts/`.

Rendering Tab

- **Encode Mode**: `Auto` (best available), `Fast` (speed priority), or `Quality` (size priority).
- **Prefer HEVC**: Use H.265 when available for smaller file sizes.
- **Image Export Defaults**: Format (PNG/JPG), size (720p / 1080p / 4K), JPG quality slider, and GPS metadata embedding.

Docs

In the desktop app, open **Help → Documentation** for the offline user guide and technical docs.

Updating the App

The desktop app keeps itself up to date automatically *(desktop only)*.

- **Automatic check on launch**: A few seconds after the app opens, it checks GitHub for a newer release. If one is found, it downloads in the background.
- **Home-screen indicator**: When an update is downloading or ready, an orange **Update** badge appears next to the version in the "Desktop Edition" pill on the home screen. Click it once the update is downloaded to restart and install.
- **Restart to install**: When the download finishes, a banner also appears in the bottom-right corner — click **Restart to install** to relaunch into the new version. (You can also dismiss it with **Later**; the update installs automatically the next time you quit the app.)

- **Manual check**: Go to **Settings** → **General** → **About** to see your installed version and click **Check for updates** at any time.

No reinstall or manual download is needed — updates apply in place.

Importing an Activity (FIT/GPX/TCX)

Supported Formats

- **FIT**: Garmin, Wahoo, and other ANT+/BLE devices — richest data (power, HR, cadence, temperature, running dynamics)
- **GPX**: Most GPS devices and apps — GPS track + elevation
- **TCX**: Garmin Training Center XML — GPS + HR + cadence + power; commonly exported by Garmin Connect and Strava

How to Import

1. Click **Start new project** on the home screen.
2. Upload your `.fit`, `.tcx`, or `.gpx` file.
3. Wait for parsing, then click **Go to Editor**.

Tips

- Use original FIT files from your device for the most complete data (power, HR, cadence, temperature).
- TCX is a good alternative when FIT is unavailable — it carries more metrics than GPX.
- Avoid heavily edited/trimmed files if parsing fails.

Importing from Strava

Connect your Strava account to browse and import activities directly — no manual FIT/GPX download required.

Setup

Desktop app

1. Open **Settings** → **Integrations** tab.
2. Enter your **Client ID** and **Client Secret** from the Strava API settings.

3. Set the **Redirect URI** to match what you registered in the Strava API settings (e.g. `http://localhost:3000/strava/callback`).
4. Click **Save Settings**.

Web mode

Set the following environment variables in `apps/web/.env.local`:

```
STRAVA_CLIENT_ID=<your_client_id>
STRAVA_CLIENT_SECRET=<your_client_secret>
STRAVA_REDIRECT_URI=http://localhost:3000/strava/callback
```

Alternatively, go to **Settings** → **Integrations** in the web UI and enter the credentials there.

Connecting

1. Click **Strava** in the editor header (or **Import from Strava** on the start screen).
2. The Strava activity picker opens. If not yet connected, click **Connect Strava**.
3. You are redirected to Strava's authorization page — approve access and you are returned to the editor.

Importing an Activity

1. Once connected, the picker shows your recent activities grouped by sport type.
2. Use the **Sport type** filter to narrow the list (Cycling, Running, etc.).
3. Each activity shows name, date, distance, and moving time.
4. Click an activity to download it. The app fetches all available streams from Strava (GPS, time, altitude, distance, heart rate, cadence, power, temperature) and converts them to a TCX file automatically.
5. The track is parsed and loaded — click **Go to Editor** to proceed.

What Data Is Available from Strava

Field	Availability
GPS (lat/lon)	Always
Altitude	Always
Distance	Always
Heart Rate	If recorded by your device and shared on Strava
Cadence	If recorded by your device

Power	If recorded by a power meter
Temperature	If recorded by your device

Tips

- Strava imports now use TCX format and include HR, cadence, power, and temperature where available — you no longer need a separate FIT file for those metrics.
- If your device recorded power or HR but the data is missing from the import, check that **Privacy controls** on the Strava activity allow data sharing.
- The editor header shows a **Strava** source badge and links to the original activity page when a Strava-sourced track is loaded.
- Click **Disconnect** in the Strava picker to revoke the stored tokens at any time.
- Rate limits: Strava allows 100 requests per 15 minutes. If you hit the limit, wait a moment and try again.

Importing from Garmin Connect

Import activities straight from Garmin Connect using Garmin's **Personal API**. Unlike the Strava import, the app downloads your activity's **original file** (normally the FIT your device recorded), so every metric your watch or bike computer logged is kept at full resolution.

Setup

1. Open the Garmin Connect Personal API page while signed in to Garmin Connect.
2. Click **Create Key**, accept the terms, and pick an expiry (30 days to 1 year). **Copy the key** — Garmin shows it only once.
3. In the app, open **Settings** → **Integrations** → **Garmin Connect**, paste the key, and click **Save & test**. "Connected" confirms Garmin accepted it.

In the desktop app the key is encrypted with your OS keychain. In web mode you can instead set `GARMIN_API_KEY=<your key>` in `apps/web/.env.local`.

Importing an Activity

1. Click **Garmin** in the editor header.
2. Browse your activities (newest first). Narrow the list with **Search** (name/description), a **from date**, or an **activity type** (Cycling, Running, …). Click **Load more** for older activities.
3. Click an activity. The app downloads the original file, parses it, and loads the track. The activity name, date, sport and location pre-fill the Header widget, and the **Project** tab shows **Imported from**

Garmin Connect with a link back to the activity.

Tips

- The Personal API is for **your own data** — each person uses their own key.
- When the key expires (Garmin emails you about a week before), create a new one and paste it in Settings again.
- Activities marked **no GPS** (indoor rides, treadmill runs) import fine, but map-based widgets stay empty.

Render Notifications (Telegram)

Get render progress pushed to a Telegram chat so you can walk away from long renders.

Setup

1. In Telegram, message @BotFather, send `/newbot`, and follow the prompts to get a **bot token**.
2. Start a chat with your new bot (send it any message), then find your numeric **chat ID** — for example by messaging @userinfobot.
3. Open **Settings** → **Integrations** → **Telegram**, paste the **Bot Token** and **Chat ID**, choose when to be notified, and click **Save**.
4. Click **Test** to confirm a "✅ Telegram connected" message arrives in your chat.

Notify modes

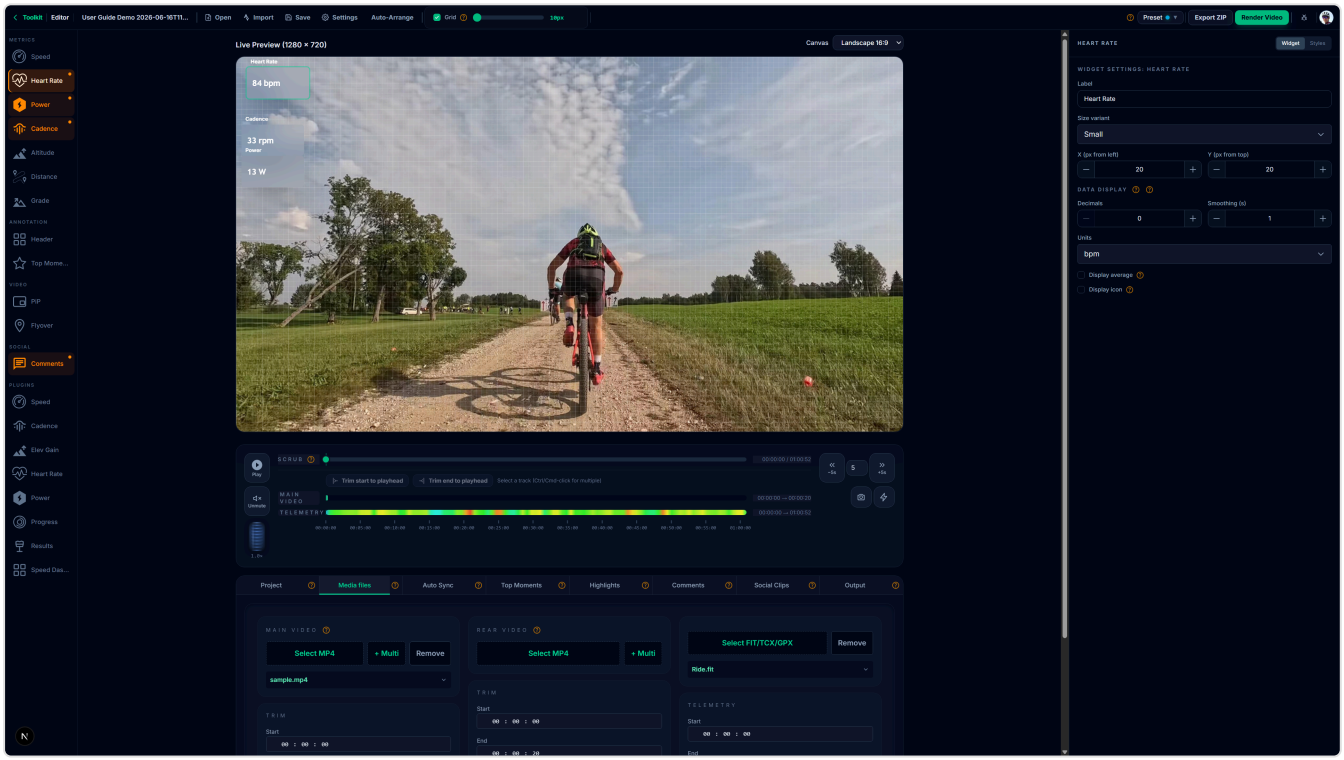
- **Milestones** (default): a message when the render starts, an update every 25% (or after each chunk for chunked renders), and a final message on completion.
- **Lifecycle**: start and completion only.
- **Completion**: a single message when the render finishes, fails, or is cancelled.

The app sends **one** message per render and edits it in place as progress advances, so your chat is not flooded. Each message shows the **project name**, the time the render **started**, and the **expected end time** (projected from progress so far, refined as the render proceeds), alongside the progress bar, frame count, and ETA. Notifications are outbound only — the bot does not accept commands. Toggle **Send render notifications** off to pause them without clearing your credentials. In the desktop app the bot token is encrypted at rest via the OS keychain.

Understanding the Editor

Layout Overview

- **Left**: Widget list (enable/disable widgets; click a widget to select it)
- **Center**: Live preview canvas (1280×720 by default; portrait / square canvases also supported — see Canvas Sizes & Square-Source Video). The preview automatically shrinks or grows to fit the space left above the bottom panel: drag the panel's top edge or collapse it (double-click the handle / chevron) to give the preview more room.
- **Right**: Widget inspector (per-widget settings or global HUD styles)
- **Bottom**: Playback controls — play / pause, scrubber, **zoom wheel** (1×–40×), **horizontal scrollbar**, and the timeline rows for Main / Rear / Telemetry tracks
- **Bottom tabs**: Media files · Auto Sync · Top Moments · Highlights · Comments · Output · Social Clips



Right Panel: Widget Inspector

The right panel adapts based on what is selected:

- **Nothing selected**: Shows the full global style editor (Card, Pill, Icon, Text styles for all variants).
- **Widget selected**: Shows a scope toggle:
  - **This widget** — position, size, and type-specific settings for the selected widget.
  - **All [style] style** — global style editor filtered to the widget's style variant.

Bottom Panel Tabs

Tab	Purpose

Media files	Load and trim Main Video, Rear Video, and Telemetry CSV; set trim start/end and offset.
Auto Sync	Automatically compute the telemetry offset by matching GoPro GPS against the FIT track.
Top Moments	Detect performance peaks (speed, power, HR, climbs, jumps) and surface them as highlights. Add a moment to the Highlights overlay, then edit its label, start time, and duration inline in the same card.
Comments	Add timed text annotations that appear over the video.
Output	Global output framing for social clips — Reel (9:16) / Feed (1:1) / Custom W×H, rotation, fit mode, pan, audio. See Output Transform.
Social Clips	Define short clips for social platforms (start time + duration + enable toggle). All clips render through the global Output Transform.

Key Actions

- **Add Widget**: Click a widget chip in the left panel to enable and select it.
- **Move Widget**: Drag on the canvas or type exact X/Y values in the right panel.
- **Preset**: Switch color themes from the editor header.
- **Export**: Save telemetry CSV + `widgets.json` for external use.

Timeline Zoom & Scrubbing

The timeline supports up to **40× zoom** so you can edit long rides (4 hours +) precisely. The zoom wheel sits at the bottom-right of the timeline; a horizontal scrollbar appears below the rows when zoomed in.

Using the Zoom Wheel

- **Drag the wheel up or down** to zoom continuously (drag distance maps logarithmically to zoom factor).
- **Scroll your mouse wheel** over the wheel for incremental steps.
- **Click the top half** to zoom in one step, **bottom half** to zoom out.
- **Double-click** the wheel (or press `0` / **Home**) to reset to fit-the-viewport.
- **Arrow keys** nudge the zoom one step at a time when the wheel is focused.

Each zoom change preserves the **time under your cursor** — wherever the pointer is when you zoom, that timestamp stays pinned to the same pixel. This makes it easy to drill into one moment of a long ride without losing your bearings.

Horizontal Scrolling

Once zoomed beyond 1×, the timeline scrolls horizontally:

- **Drag the scrollbar thumb** below the timeline rows.
- **Click the scrollbar track** on either side of the thumb to page (~90 % of the visible width).
- **Keyboard**: focus the scrollbar and use **arrows**, **PageUp/PageDown**, **Home**, **End**.
- During playback the viewport **auto-follows the playhead**, scrolling smoothly so the playhead stays in view.

Smoother Scrubbing

When you drag the playhead, the widgets and timecode update instantly while the video catches up in the background. This keeps the scrubbing UI responsive even on multi-gigabyte source files.

Speed Zone Gradient

The telemetry row is now colored by **instantaneous speed** across the visible window — blue is slowest, red is fastest. Sustained high-speed sections jump out at a glance; flat-speed sections render as plain gray.

Multi-Track Trim

You can trim the start or end of multiple timeline tracks (Main video, Rear video, Telemetry) in a single click.

1. **Select tracks** — Ctrl-click (Cmd-click on macOS) the row labels at the left of the timeline. Selected rows highlight. Click without Ctrl/Cmd to select only that row.
2. **Position the playhead** at the time you want to trim to.
3. Click **Trim Start** or **Trim End** above the timeline. The selected tracks update; unselected tracks are unchanged.

Tips

- The buttons disable when no tracks are selected or when the playhead is outside every selected track's bounds.
- Trim values are clamped: Trim Start cannot exceed the current Trim End, and vice versa.
- Selection is not saved with the project — reopening the editor clears it.
- After a multi-track trim, any social clip whose range falls outside the new bounds becomes unrenderable; review the Social Clips list and adjust or delete affected clips.

Media Files Tab

Load and align your video and telemetry files with the track.

How It Works

1. Open the **Media files** tab in the bottom panel.
2. Select the **Main Video** file (or multiple files — see Multi-Clip Video Sequences).
3. Optionally add a **Rear Video** for picture-in-picture overlays.
4. Use the timeline controls to set the **trim start** and **trim end** for each source.
5. Align playback with telemetry data so the overlay matches the footage.

Tips

- Use a recognizable moment (e.g., start of movement or a known landmark) to sync video and telemetry.
- The rear video is optional — needed only for PiP overlays.
- Use **Auto Sync** to compute the telemetry offset automatically if your main video is a GoPro.
- For HEVC GoPro clips, the editor automatically uses the `.LRV` sidecar or generates an H.264 proxy for smooth preview playback — see Video Playback (Under the Hood).

Video Playback (Under the Hood)

The editor never plays the original render-source file directly. Instead, it resolves the **lightest playable version** of each video in the background and uses that for preview. Renders always use the original source at full quality.

Resolution Order

For each video file, the editor tries:

1. `.LRV` **sidecar** — GoPro low-resolution-video file in the same folder (HERO 8+ recordings include these). Detected by filename pattern; validated for duration ± 2 s of the original.
2. `.LRF` **sidecar** — DJI low-resolution-file equivalent. Same validation.
3. **Cached H.264 proxy** — if the editor has previously generated a proxy for this source, it's reused. The cache key includes source size + modification time, so re-recording with the same filename re-generates.
4. **Background proxy generation** — a 1280-wide H.264 / CRF 28 / 15 fps proxy is generated via FFmpeg. A progress indicator appears in the video area while it runs; playback starts as soon as the proxy is ready.

When You'll Notice

- **GoPro HERO 8+ recordings** — instant playback via `.LRV`. Nothing to wait for.
- **DJI recordings with `.LRF` sidecars** — same.
- **First time opening an HEVC clip without a sidecar** — short wait (proxy generation runs at roughly real time on a modern CPU; an 8-minute clip takes ~8 minutes the first time, then is instant).
- **Subsequent sessions** — instant from the cache.

Cache Location

- **Desktop**: under the app's `userData` folder (cleared with the app's Reset action).
- **Web**: under `apps/web/.next/cache/media-proxies/`.

Limitations

- **Multi-clip projects** still use the older stitching flow for playback. Migration to the new resolver is planned (Milestone 4).
- The browser is never given the original-source path — only an opaque preview ID. This is a deliberate security boundary; do not rely on guessing media URLs in your scripts.

Multi-Clip Video Sequences

The editor supports multi-clip video sequences for both main and rear camera inputs. Load all chapter files from a split GoPro recording and the renderer stitches them transparently before encoding.

There are two ways to work with split recordings: **Combine Clips** up front (recommended), or load them as an in-editor sequence.

Combine Clips Before Editing (Recommended)

On the **home screen** (desktop app), click **Combine Clips** to stitch your chapters into a single file *before* you start editing. This is the simplest workflow for split GoPro recordings:

1. Click **Combine Clips** on the home screen.
2. **Add main camera clips** (and optionally rear camera clips) — select all chapters at once. Reorder with the ↑ / ↓ buttons; remove with ✕. Each clip shows its resolution/codec **and whether a GoPro `.LRV` or DJI `.LRF` sidecar was detected** (used to build a fast preview).
3. Choose whether to **keep audio** from the main camera.
4. Click **Combine clips**. The app first checks the output drive has enough free space (the combined file is roughly the size of all the clips added together) and warns you if it doesn't. It then concatenates each camera's chapters into one file (lossless `-c copy`, fast). It also stitches the matching

`.LRV` / `.LRF` sidecars into a single low-resolution preview proxy (or generates one if some clips lack a sidecar).
5. When it finishes, optionally **Move originals to Recycle Bin** to reclaim disk space (the combined files are kept; this is recoverable from the Recycle Bin).
6. Click **Open in editor** — the editor opens on the single combined files.

Why combine first?

- **Re-render freely.** Because the editor works on a single file, re-rendering with different settings never re-stitches the source clips.
- **Simpler trimming & image export.** Everything operates on one continuous file.
- **Reclaim space.** Once combined, you can delete the original chapter files and keep just the stitched output.

The combined files are written to a `stitched/` folder inside your input directory. Combining is a desktop-only feature.

How to Add Multiple Clips (In-Editor Sequence)

1. In the **Media files** tab, click **+ Multi** next to the Main Video or Rear Video selector.
2. Select all chapter files at once in the file dialog.
3. A badge shows the number of clips loaded (e.g., **4 clips**).
4. Click **Manage clips (4)** to open the clip manager.

Manage Clips Modal

- **Reorder**: Drag clips to change their order.
- **Metadata**: Each clip shows duration, resolution, fps, and codec.
- **Running total**: The footer shows the combined duration of all clips.
- **Incompatible clips**: A warning is shown if a clip's resolution or fps differs from the first clip.
- Click **Save** to commit the order, or **Cancel** to discard changes.

Why Use Multi-Clip

- GoPro cameras split long recordings into 4 GB chapter files. Load all chapters as a sequence for a single continuous render.
- Stitching is lossless (`-c copy`) and fast — a 40-minute, 4-chapter ride stitches in seconds.
- Existing single-video projects load without changes.

Playback for Multi-Clip Projects

Single-file projects use the new sidecar-or-proxy resolver described in Video Playback (Under the Hood). **Multi-clip projects still use the legacy stitch-and-preview flow** — the editor builds a temporary stitched

preview the first time you load the project. Migration of multi-clip playback to the new resolver is planned for a future release.

Canvas Sizes & Square-Source Video

VeloOverlay supports output canvas presets beyond the default 16:9 widescreen, which is useful when your source footage has a non-standard aspect ratio (e.g. GoPro square sensor or portrait crop).

Canvas Presets

Preset	Resolution	Use case
16:9	1280 × 720	Standard widescreen (default)
Square	720 × 720	Instagram square posts
4:3	960 × 720	GoPro wide/linear square sensor clips
9:16	405 × 720	Portrait (Instagram Reels, TikTok)

Square-Source Detection

When you load a GoPro clip recorded in **Wide** or **Linear** mode at a square sensor resolution, the editor automatically detects the native aspect ratio and suggests a matching canvas preset. Accept the suggestion or pick a different preset from the dropdown in the editor header.

Crop Workflow

1. Load your video in the **Media files** tab.
2. If the editor detects a non-standard source, a preset suggestion appears — click **Apply** to switch the canvas.
3. You can also manually select a preset from the **Canvas** dropdown in the editor header at any time.
4. Position and resize widgets within the new canvas boundary. The design space always maps to the selected output resolution.
5. Render as usual — the output file will have the chosen dimensions.

Tips

- Changing the canvas preset repositions widgets proportionally where possible.
- For social clips in portrait orientation, combine the **9:16** canvas with the **Social Clips** tab to define clip segments.

- See `docs/square-source-video.md` for additional crop workflow details and the full list of supported presets.

GoPro Auto Sync

Automatically compute the telemetry start offset by matching GoPro GPS positions against the FIT track — no manual scrubbing required.

Requirements

- A GoPro video loaded as the main video (must contain embedded GPS data).
- A FIT file loaded as telemetry.

How to Use

1. Click the **Auto Sync** tab in the bottom panel.
2. Click **Extract GPS from Video** and wait for the sample count to appear.
3. Click **Auto-sync** — the app matches GPS positions using haversine minimization.
4. Review the result: **offset (seconds)**, **confidence** (High / Medium / Low), and **mean GPS distance (m)**.
5. Click **Apply Offset** to write the value to the telemetry timeline.
6. Verify alignment on the canvas before rendering.

Confidence Levels

Confidence	Mean GPS Error	Interpretation
High	< 20 m	Tracks overlap cleanly — offset is reliable.
Medium	20–100 m	Usable, but verify on the canvas.
Low	> 100 m	Tracks may not cover the same route segment at all.

Tips

- Extracted GPS is cached server-side for the session — re-running sync costs no extra extraction time.
- The search window is ±180 seconds by default, which covers most alignment scenarios.
- Low confidence usually means the GoPro GPS and FIT file cover different parts of the activity. Check that you loaded the correct files.

Working with Widgets

Widgets are the building blocks of your overlay. Each widget is independently positioned, sized, and styled on the 1280×720 design canvas.

Telemetry Widgets

Display live data values from your FIT/GPX track. All are enabled by default.

Widget	What it shows
Speed	Current speed with optional rolling average
Heart Rate	Current HR with optional average
Cadence	Current cadence with optional average
Power	Current power output with optional average
Grade	Current road gradient (%)
Distance	Cumulative distance
Altitude	Current elevation

Each telemetry widget supports: position (X/Y), preset size (Small / Medium / Big), unit selection, decimal precision, smoothing, and optional icon / average display.

Visualisation Widgets

Disabled by default — enable them from the widget list.

- **Progress Bar**: A full-width bar showing route or time progress. Includes an elevation silhouette, coloured segments for climbs and jumps, a glowing current-position indicator, and configurable labels. See the widget settings panel for bar style, colors, and label options.
- **Results Card**: A summary card shown at a fixed time in the video (e.g., the finish). Displays total distance, moving time, elevation gain, average speed, average HR, and average power. Configure metrics and card style in the settings panel.

Media Widgets

Disabled by default — enable them from the widget list.

- **Picture-in-Picture (PiP)**: Overlays your rear camera footage as a resizable rectangle. Requires a rear video to be loaded in the Media files tab. Position and size are set in the widget settings panel.

- **Flyover Map**: Generates an animated map flyover from the GPS route and composites it as a video overlay. See the Flyover Map section for details.

Plugin Widgets

Advanced widgets delivered via the widget-plugins package. Disabled by default — enable from the widget list.

- **Header**: A branded header bar that spans the top of the canvas. Configurable fields include sport icon, route name, distance, and elapsed time. Supports four built-in layout templates (broadcast lower-third, compact pill, minimal bar, route plate) plus full color and typography overrides.
- **Metric v2**: A redesigned telemetry metric display with glass-effect backgrounds, fine-grained unit and precision control, and support for all telemetry fields (speed, HR, cadence, power, grade, distance, altitude). Use it as a replacement for the standard telemetry widgets when you want a more polished look.

Annotation & Event Widgets

- **Comments**: Timed text annotations that appear at specific moments. Enabled by default. See the Comments section.
- **Top Moments**: Automatically detects and highlights the best moments from your activity — speed peaks, power surges, HR zones, climbs, and jumps. See the Top Moments section.

Weather Widgets

- **Estimated Wind**: A themed plugin (in the Plugins list) showing estimated headwind / tailwind / crosswind from historical weather, in compact-card or compass-dial mode. Requires fetching wind history first. See the Estimated Wind section.

Widget Settings

All widgets share these common controls:

- **Position** (X/Y): drag on the canvas or type exact values
- **Preset size**: Small / Medium / Big (sets default dimensions)
- **Enabled / disabled**: toggle per widget

Icon Picker

Telemetry widgets that display an icon (Speed, Heart Rate, Cadence, Power, etc.) now support multiple icon variants that you can select directly from the editor.

How to Use

1. Select a widget that supports icons in the left panel.
2. In the right panel, enable **Display icon**.
3. Click **Choose icon** (or **Change icon** if one is already set).
4. The icon picker opens and shows all available variants for that widget type.
5. Click a variant to preview it themed with the widget's current color.
6. Click **Apply** to set the icon, or **Cancel** to close without changes.

How Variants Work

Icons are stored in per-type directories under `public/icons/`:

Widget type	Available variants (examples)
Speed	`default`, `filled`
Heart Rate	`default`, `filled`, `pulse`
Power	`default`, `flash`, `flash_filled`
Cadence	`default`, `filled`
Others	`default`

All icons are color-normalized so they pick up the widget's theme color automatically in both preview and final render.

Comments

The comments panel lets you add timed text annotations that appear over the video during playback and rendering.

How to Use

1. Open the **Comments** tab in the bottom panel.
2. Click **Add Comment** to create a new entry.
3. Set the **start time** (hh:mm:ss) and **duration** for each comment.
4. Type the comment text — it will be displayed on the overlay at the specified time.
5. Use **Set to current time** to quickly align a comment with the current playback position.
6. Use **Navigate to time** to jump the preview to a comment's start.
7. **Duplicate** creates a copy of an existing comment for quick editing.

Tips

- Keep comments short for readability on screen.
- Use the profile image (set in Settings) to personalize the comments widget appearance.

Output Transform

Output Transform sets the **global framing** applied to every Social Clip during render — one decision for the whole project instead of per-clip configuration. Reels, Feed posts, and custom aspect ratios all live here.

How to Use

1. Open the **Output** tab in the bottom panel.
2. Toggle **Enable Output Transform** on. When off, clips render at their source resolution with no transformation.
3. Pick a **Preset**:
    - **Instagram Reel** — 1080 × 1920 (9:16 portrait)
    - **Instagram Feed** — 1080 × 1080 (1:1 square)
    - **Custom** — type your own width and height
4. Set the **Rotation** — `0°`, `+90°`, or `−90°`.
5. Choose a **Fit Mode**:
    - **Cover** — crop the source to fill the output (no letterboxing). The pan controls shift the crop window.
    - **Contain** — letterbox the source so the whole frame is visible. The pan controls shift the video inside the canvas.
6. Use the **Pan X** and **Pan Y** sliders (−1 to +1) to offset the crop or letterbox. **Reset pan** centers both.
7. Toggle **Include audio** to keep or strip the audio track.

Tips

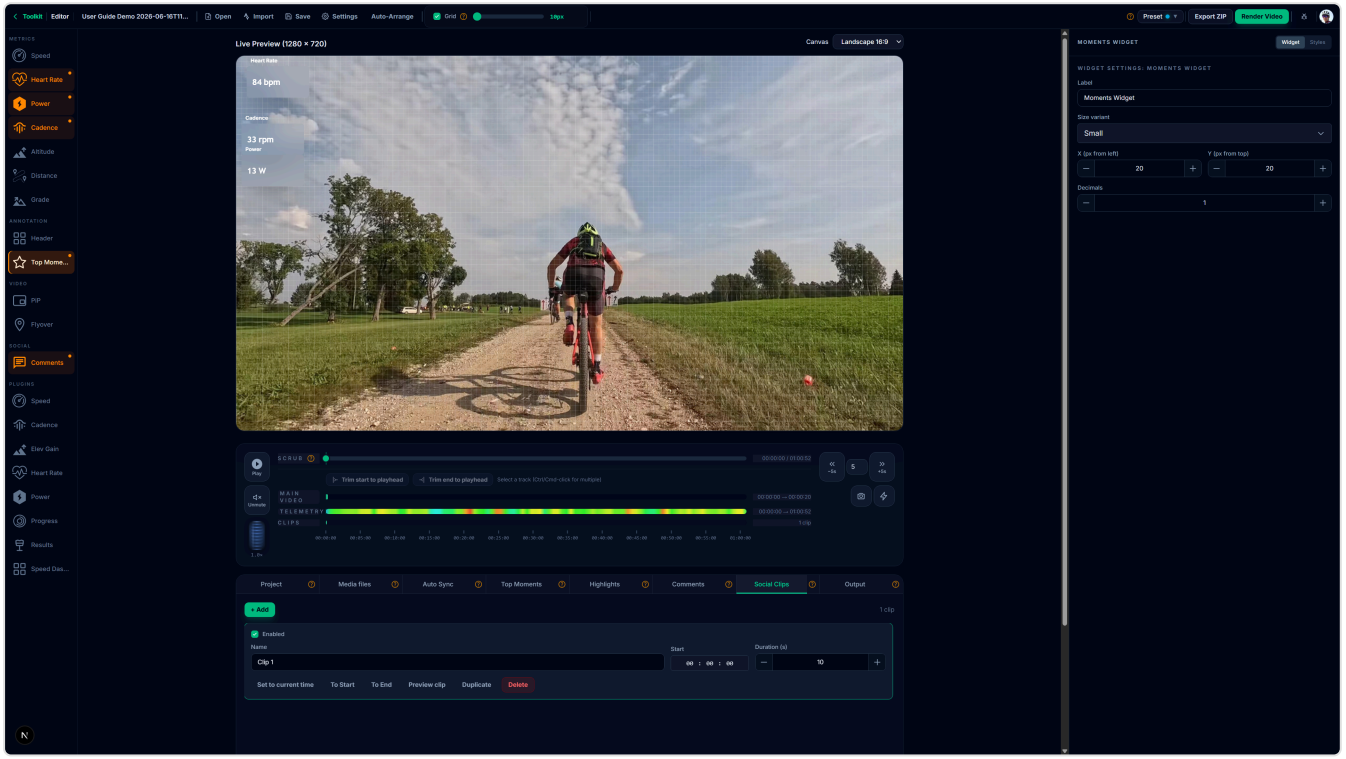
- Output Transform applies uniformly to every clip in the Social Clips list. To exclude a clip, toggle its **Enabled** switch in the Social Clips tab.
- HUD widgets are drawn at the final output size — corner-anchored widgets (like Header) re-position correctly when you switch aspect ratios; you do not need to re-place them.
- Settings here are saved with the project.

Migration from earlier versions

If you open a project saved before this version, per-clip framing fields (target resolution, rotation, fit mode, pan) on individual clips are silently dropped at load time. Use the global **Output Transform** to recreate the framing you had.

Social Clips

Social Clips define short time-ranges within your project that are exported as standalone short videos — typically for Instagram Reels, Feed posts, or other social platforms.



How to Use

1. Open the **Social Clips** tab in the bottom panel.
2. Click **Add Clip** to define a new segment.
3. Set the **clip name**, **start time** (hh:mm:ss), and **duration** for each clip.
4. Toggle **Enabled** on a clip to include it in the next render; toggle off to keep the definition but skip it.
5. Use **Set to current time** to set the clip start from the current playback position.
6. Use **Navigate to time** to preview a clip's starting point.
7. **Duplicate** a clip to quickly create variations.

Generate a Story Automatically

Instead of placing clips by hand, the **Generate story…** button (top of the Social Clips tab, available once telemetry is loaded) builds a "video story" from the fastest sections of your GPS track:

1. Click **Generate story…** and choose the **number of clips**, the **clip duration** in seconds, and the **minimum average speed** (in your configured speed unit).

2. Click **Generate**. The app finds the fastest clip-length sections whose average speed meets the minimum, spreads them across the ride (at least 25 seconds apart), and appends them to the clip list in chronological order.
3. Each generated clip is titled with its rank and average speed (e.g. `Story #1 · 48.3 km/h`). If fewer sections reach the minimum speed than requested, the remaining slots are filled with the next-fastest sections, marked **(below min)** — delete any you don't want.

Existing clips are never modified; generated clips are regular social clips and can be renamed, moved, trimmed, or disabled like any other. Combine them with **Combine clips into one video** to export the story as a single video.

Framing

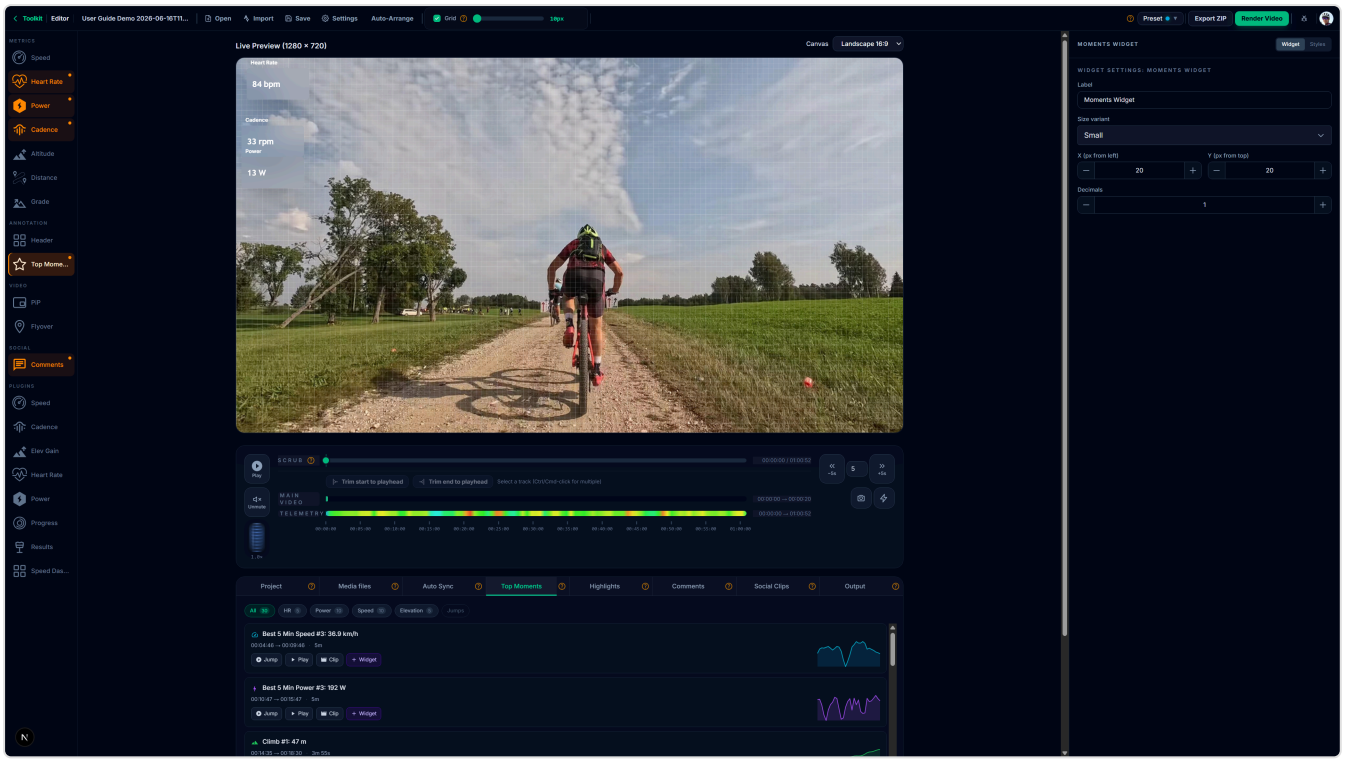
Framing is **global** — resolution, rotation, fit mode, and pan are configured once in the **Output** tab (see Output Transform) and applied to every clip. Per-clip framing is no longer supported.

Tips

- Use the **Output** tab to set up Reel-style portrait framing once; all your clips render the same way.
- Keep clips under 60 seconds for most social platforms.
- Each clip is rendered as a separate output file during the render step.
- After a multi-track trim (see Multi-Track Trim), check that no clips fall outside the new bounds.

Top Moments

The Top Moments widget automatically detects the best moments in your activity — speed peaks, power surges, HR zones, climbs, and jumps — and displays a brief animated highlight card during those windows.



How to Use

1. Enable the **Moments Widget** in the widget list.
2. Open the **Top Moments** tab in the bottom panel.
3. Choose which **metrics** to detect (speed, power, HR, climbs, jumps) and how many highlights to surface.
4. Click **Detect Moments** — the app analyses your telemetry and populates the list.
5. Review the detected moments in the list. You can remove individual items or adjust their labels.
6. Click **Widget** on any moment to add it to the Highlights overlay. The card then reveals inline controls to edit that marker's **label**, **start time**, and **duration** (or **Remove** it from the overlay).

Tips

- Moments are detected from your telemetry data, so a richer FIT file (with power, HR) produces more varied highlights.
- Adjust the **maximum number of moments** to keep the overlay from becoming crowded.
- Detected moments are saved with the project and do not need to be re-detected after reopening.

Flyover Map

The Flyover Map widget renders directly inside the normal preview and export pipeline. It can show a clean route-only view or an optional raster tile map background under the route.

Requirements

- Your track must contain GPS data (lat/lon). FIT files from GPS-enabled devices and all GPX files qualify.

How to Enable

1. Toggle the **Flyover Map** widget in the widget list (disabled by default).
2. The widget appears immediately on the preview canvas and updates live as you scrub or play.

Settings

Open the widget settings panel to configure:

Setting	Description	Default
Mode	**Follow Camera** for an immersive moving view, or **Fit Route** for a stable overview	Follow Camera
Rotate / North Up	Follow the rider heading or force a fixed north-up orientation	Rotate on
Width × Height	Widget dimensions in pixels	400 × 225
Map Background	Enable raster tile imagery under the route	On
Map Provider	Use the built-in OpenStreetMap tile template or a custom raster tile URL template	OpenStreetMap
Zoom Offset	Bias the automatically selected tile zoom	`0`
Fallback Color	Background color used when tiles are disabled or unavailable	`#0f172a`
Completed / Remaining Color	Route trail colors	`#ff5500` / slate
Marker Color	Color of the moving position marker	`#ffffff`

Rendering Behavior

The flyover widget is drawn directly by the renderer — no separate generation step:

1. In the editor, the flyover updates live in preview.
2. During export, required map tiles are preloaded before rendering starts.
3. The flyover widget is composited at the lowest widget layer, so other widgets can sit on top of a full-screen flyover.

Tile Cache Controls

The widget settings panel includes a **Tile Cache** section:

Control	Description
Cache info	Shows how many tiles are stored on disk and the total size
Refresh	Reload the current cache stats
Clear Cache	Delete all cached tiles from disk — they will be re-fetched on next use

Use **Clear Cache** when you switch to a different map provider or want to free up disk space.

Tips

- Use **Follow Camera** for a cinematic full-screen map and **Fit Route** for a PiP overview.
- If you work offline, previously cached tiles are reused automatically.
- If tiles are unavailable, the route still renders over the fallback background color.

Estimated Wind

VeloOverlay can estimate whether you faced a **headwind**, **tailwind**, or **crosswind** during an activity and show it on the overlay. Wind is **estimated from historical weather** (the Open-Meteo Historical Weather API), not measured by an on-bike sensor — so treat it as a model estimate, not exact wind.

Requirements

- The activity must have **GPS coordinates** and **timestamps** (any FIT/GPX/TCX import with a recorded route and start time).
- An internet connection is needed **once**, to fetch the wind history. After that the data is saved with the project and renders work offline.

Fetching wind history

1. Open the **Wind** tab in the bottom panel.

2. Click **Fetch Historical Wind**. VeloOverlay samples a handful of points along your route (up to 25), requests hourly wind for the activity's date, and computes head/tail/crosswind from your GPS heading.
3. When it finishes you'll see a summary: provider, date, sample counts, average wind, and the strongest head/tail/crosswind. Use **Refresh wind data** to re-fetch or **Clear wind data** to remove it.

> **Privacy:** fetching sends sampled route coordinates and the activity date to Open-Meteo. Wind is never fetched automatically — only when you click Fetch.

Adding the widget

Add the **Estimated Wind** widget from the **Plugins** list in the Widgets panel. It has two modes (Widget Inspector → **Mode**):

- **Compact card** — a glass card with a small compass dial, the current classification, the component (or wind) speed, and the wind direction, e.g. `HEADWIND · 12 km/h · from NW`.
- **Compass** — a large compass dial: you (the rider) always face up, with a colored arrow showing where the wind is coming from and motion streaks for its strength.

The dial shows a top-down cyclist with the wind arrow colored by type (headwind, tailwind, or crosswind). The widget follows your chosen **HUD theme** (Cyberpunk, Retro, Glass, etc.). The preview and the final render use the same wind values. Activities without wind data simply show a muted "no data" state — they never block rendering.

Limitations

- Historical wind is hourly and modelled; real wind felt on the bike varies with terrain, trees, buildings, gusts, and drafting.
- Very recent activities may not have published weather yet.

HUD Presets & Theme Links

The editor includes a preset-driven HUD styling workflow.

Presets

- Choose from built-in preset themes (for example Carbon, Steel, Garmin, Strava, and light variants).
- Presets define coordinated colors for card background, borders, primary/secondary text, and accent/icon tokens.

Apply Modes

- **New only**: Keep existing styles as-is; use preset behavior only for newly linked fields.
- **Preset-linked only**: Update only fields currently linked to the selected preset.
- **Overwrite all**: Re-link all themed color fields to preset tokens and apply the selected preset globally.

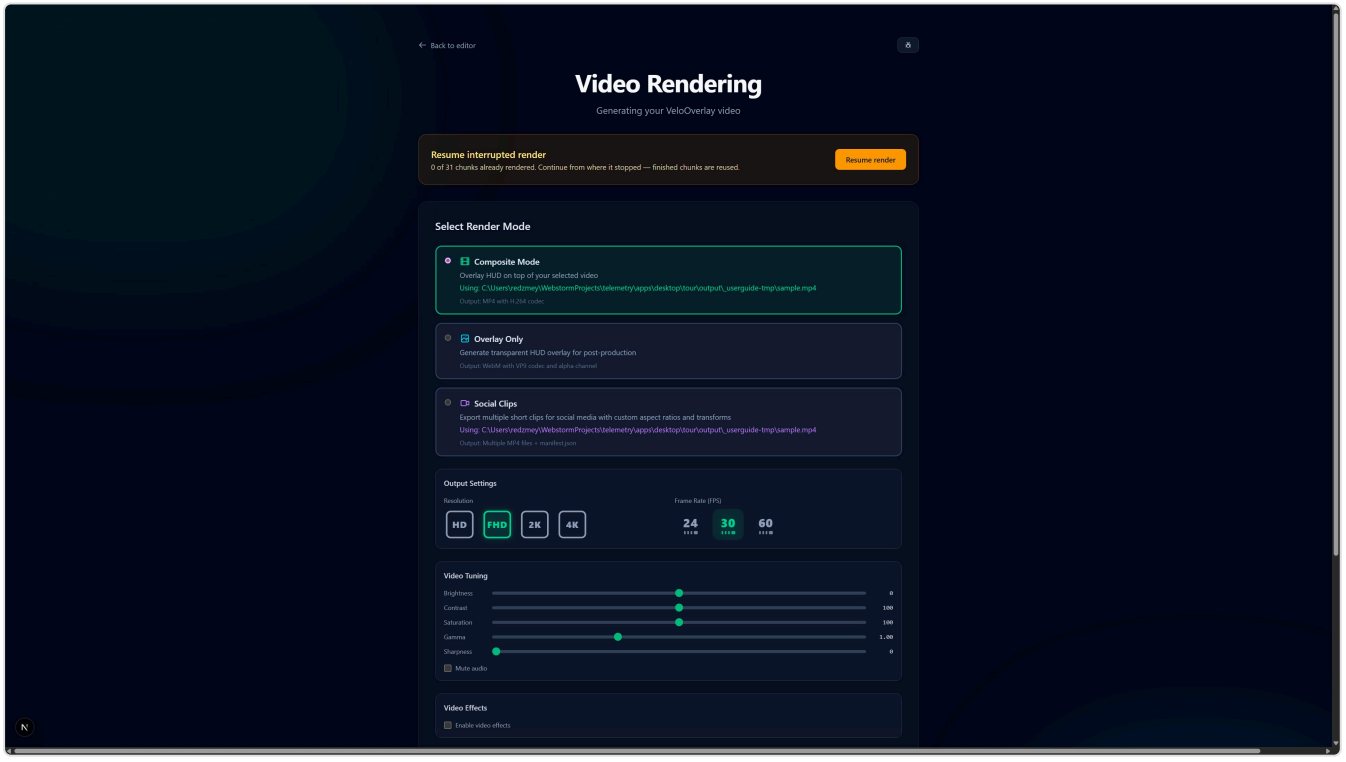
Linked vs Custom Colors

- Linked fields follow the active preset when you switch themes.
- Custom fields stay fixed and are not changed by preset switches.
- Use linked fields for global consistency, then override selected fields as custom when needed.

Rendering Videos

Render Modes

- **Overlay**: Transparent WebM (for video editors)
- **Composite**: Final MP4 with the overlay on top of your footage



Video Tuning

Before starting a render you can apply real-time color corrections to your footage. In the render page, expand the **Video Tuning** panel:

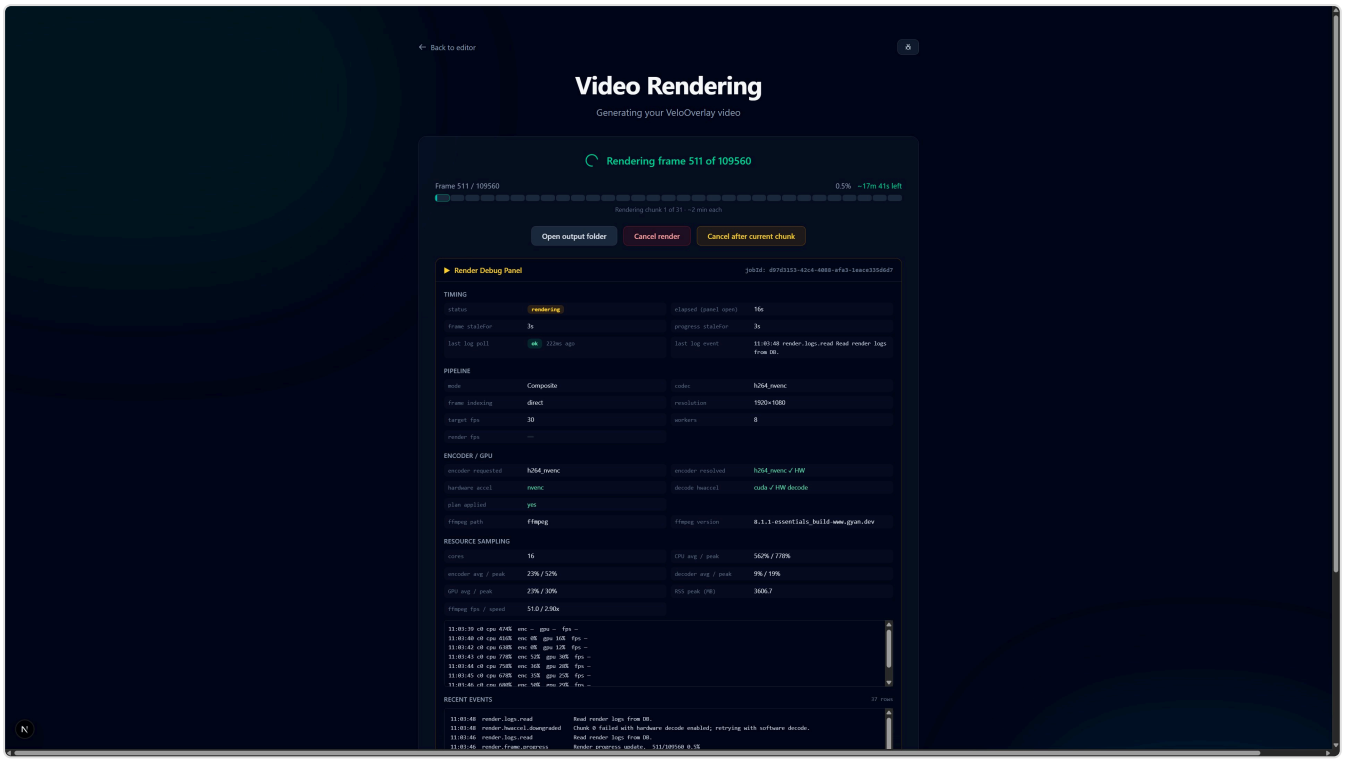
Control	Range	What it does
Brightness	−100 → +100	Overall exposure offset
Contrast	0 → 200	Tonal range expansion/compression
Saturation	0 → 200	Color intensity (0 = greyscale)
Gamma	0.50 → 2.00	Midtone brightness curve
Sharpness	0 → 100	Edge enhancement (unsharp mask)
Mute audio	on/off	Strip audio from the main track

- If a **rear video** is loaded, use the **Main / Rear / Both** toggle to apply adjustments independently or together.
- Click **Copy main → rear** to apply the same corrections to the rear track.
- Sliders with a ↺ button next to them have been changed from their default — click it to reset that value.
- Click **Reset defaults** to restore all adjustments at once.

All adjustments are applied by FFmpeg during the encode pass and are saved with the project.

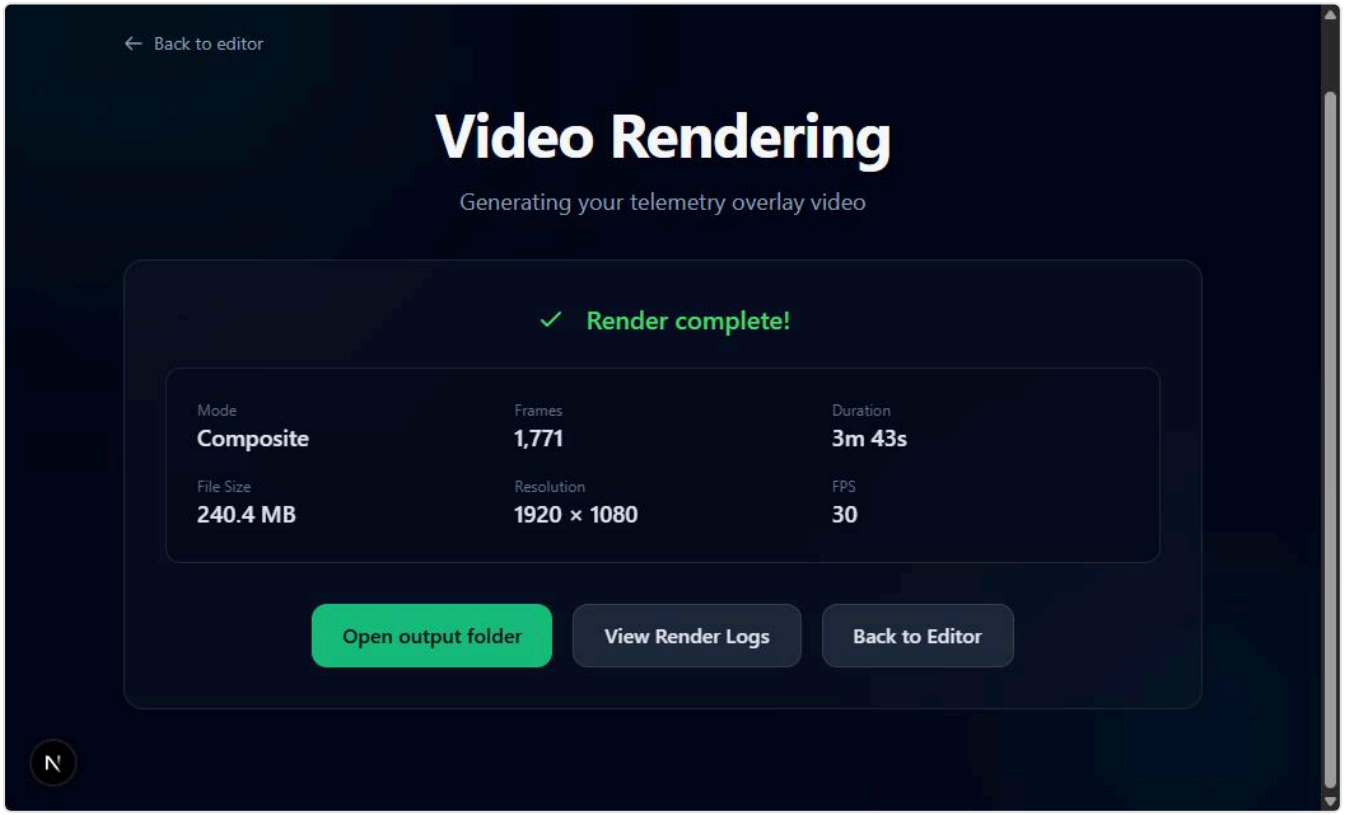
Start a Render

1. Click **Render Video** in the editor.
2. Choose mode, resolution, FPS, and encoding.
3. If using Composite, select main and rear videos.
4. Optionally adjust **Video Tuning** before rendering.
5. Click **Start Render**.



After Render

- Use **Open output folder** in the Render page to reveal the file.
- The default output location is configurable in Settings.



Encoder Auto-Selection

When the requested encoder is `libx264` (the default), the renderer probes the system at startup and may automatically upgrade to a **hardware encoder** for faster encoding:

- **NVIDIA** GPUs → NVENC (h264_nvenc / hevc_nvenc)
- **Intel** integrated GPUs → QSV (Quick Sync)
- **AMD** GPUs → AMF

The renderer only auto-upgrades to *hardware* encoders. It never silently switches to a different software codec (e.g. it will never pick libx265 unless you ask for it). To disable auto-upgrade entirely, set the encoder explicitly in **Settings** → **Rendering** or in the Render dialog.

The chosen encoder, the probe result, and any fallback reason are visible in the **Render Debug Panel** described below.

Render Debug Panel

If a render seems stuck or you want to verify the encoder it picked, enable the diagnostic panel:

1. Open **Settings** → **Diagnostics**.
2. Toggle **Show Render Debug Panel** on.
3. Start a render. The panel appears below the standard progress UI on the Render page.

The panel surfaces:

- **Timing** — elapsed time since the last frame was emitted (`staleFor`). A "possible hang" pill highlights when nothing has moved for more than 5 seconds during the rendering phase.
- **Pipeline** — mode (Composite / Clips / Overlay), codec, FPS, resolution, worker count.
- **Encoder** — which encoder was requested, which one is actually being used, whether the auto-upgrade picked a hardware encoder, and if a fallback occurred, the exact FFmpeg error.
- **Log tail** — the last 30 structured log events, pretty-printed.
- **Raw payload** — the full progress JSON, collapsed.

Render logs are stored in the app's local database (not a file). When filing a render bug, export them via **Settings** → **Diagnostics** → **Export Diagnostic Bundle**.

Uploading to YouTube

Publish a finished render straight to your YouTube channel — either with an **Upload to YouTube** button after the render completes, or automatically via a per-render checkbox.

Setup (one-time)

YouTube uploads use your own (free) Google Cloud project:

1. Go to the Google Cloud Console and create a project.
2. Under **APIs & Services** → **Library**, enable the **YouTube Data API v3**.
3. Under **APIs & Services** → **Credentials**, create an **OAuth client ID** of type **"Desktop app"** and copy the Client ID and Client Secret. (The Desktop type is important — it lets the sign-in redirect work on any local port, so you can leave the Redirect URI field in the app blank.)
4. Under **OAuth consent screen**, add your Google account as a **test user**.
5. In VeloOverlay, open **Settings** → **Integrations** → **YouTube**, paste the Client ID and Secret, pick a default visibility, and click **Save**. In the desktop app the secret is encrypted at rest via the OS keychain.
6. Click **Connect channel** — a Google sign-in popup opens; choose the account or brand channel to upload to and approve access. The connected channel name appears in Settings. To switch channels, disconnect and connect again.

Manual upload

After a render completes, an **Upload to YouTube** button appears next to the other actions on the render page. Clicking it opens a dialog prefilled with a title (from your activity name and date), a description (sport, location, duration, resolution), and the visibility — edit anything, then click **Upload**. A progress bar tracks the upload; when it finishes you get a direct **View on YouTube** link. Failed uploads show the error with a **Retry** button, and an in-flight upload can be cancelled.

Automatic upload

When a channel is connected, the render page shows an **"Upload to YouTube when done"** checkbox above the Start Rendering button. Tick it and the upload starts by itself the moment the render completes, using the generated title/description and your default visibility. The choice sticks for future renders, and it survives app restarts mid-render — a resumed chunked render still uploads when it eventually finishes. Cancelled or failed renders are never uploaded.

What can be uploaded

Uploads work for renders that produce a **single video file**: overlay and composite renders, and Social Clips renders with **Combine clips into one video** enabled. When a clips render produces multiple separate files, the upload button is not offered — combine the clips or upload them manually.

Good to know

- **Visibility while unverified:** until your Google Cloud project passes Google's verification, YouTube forces uploads from it to **Private** regardless of the visibility you choose. For personal use this is normal — the video uploads fine; you can change its visibility manually in YouTube Studio.

- **Daily quota:** each upload costs 1,600 units of the API's default 10,000/day quota — roughly 6 uploads per day.
- **Interrupted uploads:** if the app restarts mid-upload, the upload is lost (the rendered file is not) — just click Upload again.

LUT Color Correction

Apply a `.cube` color look-up table (LUT) to the composited output for cinematic grading — directly inside the FFmpeg render pipeline, with no round-trip to an external editor.

Setup

Desktop app

1. Open **Settings** → **Paths**.
2. Set the **LUT Library Folder** to the directory containing your `.cube` files.
3. The app scans this folder and makes all found LUTs available in the Video Tuning panel.

Web mode

Set the `TELEMETRY_LUTS_DIR` environment variable in `apps/web/.env.local` to your `.cube` folder path. If unset, `public/luts/` inside the app is used as a fallback.

Applying a LUT

1. Go to the **Render** page.
2. Expand **Video Tuning**.
3. Select a LUT from the **LUT File** dropdown. Only `.cube` files found in the configured library folder are listed.
4. The LUT is applied globally to the composited video during encoding — it does not affect the editor preview.

Tips

- LUTs work best on log-profile (D-Log, V-Log, S-Log) footage. Applying a LUT to standard color footage may produce unexpected results.
- The LUT is applied after all Video Tuning adjustments (brightness, contrast, etc.).
- To remove a LUT, select **None** from the dropdown.

Exporting Images

Export a single high-resolution image of the current frame with all your telemetry widgets. Useful for sharing ride snapshots on social media without rendering a full video.

How to Export

1. Open the editor with a track loaded and scrub to the frame you want to capture.
2. Click **Export Image** in the right-hand actions sidebar.
3. A modal appears with resolution controls:
    - **Quick presets**: 1080p, 2K, and 4K. Each preset's pixel dimensions are derived from the **project's canvas aspect ratio**, so a 9:16 portrait project renders 4K as 2160×3840, and a 1:1 square project renders 4K as 2160×2160.
    - **Custom dimensions**: type any width and height. The aspect lock is not enforced for custom values.
4. Click **Export PNG**. The modal closes immediately; the export runs in the background. A toast notification confirms success (with the saved file path) or surfaces an error if anything goes wrong.

What's Included

- The main video frame at the scrubbed timestamp (if a video is selected).
- Rear camera PiP overlay (if a PiP widget is enabled).
- All enabled telemetry widgets rendered at the chosen resolution.

Tips

- Preset labels update dynamically to show the resolved W × H for your project's aspect — don't be surprised if "4K" reads 2160×3840 on a portrait project.
- If no video is selected, the export contains only the widget overlay on a transparent background, which you can composite onto any image in an external editor.
- Larger resolutions take longer to generate since the server renders the full frame at that size.

Managing Output & Logs

Output Files

- **Overlay** outputs: `.webm`
- **Composite** outputs: `.mp4`

Logs

- Open **Settings** → **Paths** to see where logs are stored (Logs Directory).
- Render page includes **View render logs** for job-specific troubleshooting.

Tips & Best Practices

- Use 1080p/30fps for most projects.
- Keep widgets readable: 3–6 widgets is a clean layout.
- Use the **Preview** to spot layout issues before rendering.
- If a render is slow, try lower resolution or GPU encoding.
- Use a full-screen flyover when you want a map-first sequence, then layer metric widgets above it.
- For GoPro multi-chapter recordings, add all chapter files with **+ Multi** and verify order in **Manage clips** before rendering.

Troubleshooting

"Upload failed: Internal Server Error"

- Check the **Logs Directory** (Settings → Paths) for `web.log`.
- Try re-exporting your FIT/GPX file from the original device/app.

Missing video options

- Ensure **Default Input Folder** is set in **Settings** → **Paths** and contains the videos.
- The editor reads available files from the input folder.

FFmpeg errors

- Set the **FFmpeg Path** in **Settings** → **Paths**.
- Verify `ffmpeg -version` runs in your terminal (optional).

Render fails or hangs

- Open **View render logs** on the Render page.
- Check disk space in the output folder.

GoPro HEVC video won't play in the editor

Chrome and Firefox on Windows/Linux cannot decode HEVC/H.265 without extra codecs.

- If a `.LRV` sidecar file exists next to the clip (GoPro HERO 8+), the editor uses it as an instant proxy — no waiting.
- If no sidecar exists, the editor generates an H.264 proxy automatically. A progress indicator appears in the video area. Playback starts when the proxy is ready.
- The proxy is only used for preview; renders always use the original HEVC file at full quality.
- For details, see Video Playback (Under the Hood).

Timeline scrubbing feels laggy

- Make sure you're playing the preview proxy (look for the "HEVC source — using H.264 proxy" indicator if you're on a GoPro file). If the original HEVC is being played directly, decoding overhead is the bottleneck.
- The timeline scrub controller decouples widget updates from video seeks — widgets stay responsive even if the video element is slow. If the widgets themselves feel laggy, file a bug and include a Render Debug Panel snapshot.

Hardware encoder (NVENC / QSV / AMF) not detected

- Enable the Render Debug Panel and start a render. The Encoder section shows the probe result and the FFmpeg error if any.
- On NVIDIA cards, the probe runs against a 320×240 test frame because NVENC silently fails on frames below ~128×128. If you see a "Frame Dimension" error, your driver may be out of date — update GPU drivers and restart the app.
- To force CPU encoding (e.g. for debugging), set the encoder explicitly in **Settings** → **Rendering**.

Proxy generation stalls or never finishes

- Check the Render Debug Panel or `web.log` for FFmpeg errors.
- Delete the partial file from the proxy cache directory (see Video Playback for paths) and retry.
- Make sure your output disk has room — proxies are ~1/5 to 1/10 the size of the source but still substantial for long clips.

Flyover map tiles are missing

- Ensure your track contains GPS data (latitude/longitude).
- Check that the selected tile provider URL is reachable.
- If you are offline, use cached tiles or disable the map background for a route-only flyover.

Auto Sync returns low confidence

- Low confidence (> 100 m mean GPS error) usually means the GoPro video and FIT file cover different route segments or time windows.
- Check that you loaded the correct FIT file for the activity recorded on the GoPro.
- Try adjusting the trim start in the **Media files** tab before re-running Auto Sync.

Estimated Wind is unavailable or shows "no data"

- The **Wind** tab says wind needs GPS coordinates and timestamps — make sure the imported activity has a recorded route and a start time (most FIT/GPX/TCX files do).
- "Historical wind could not be fetched" usually means no internet connection, or the activity is too recent for published weather. Check your connection and try **Fetch Historical Wind** again later.
- The widget shows "no data" until you fetch wind history in the **Wind** tab. After fetching, the data is saved with the project and renders offline.

FAQ

Q: Where are my settings stored? A: In the desktop app user data directory. You can find it via the Logs Directory path (Settings → Paths).

Q: Does the desktop app work offline? A: Yes. All parsing and rendering is local. The Flyover Map can reuse cached tiles offline, or render route-only without tiles. Auto Sync requires the video to be on disk (no internet needed).

Q: Where do rendered videos go? A: The **Default Output Folder** set in Settings → Paths. Use **Open output folder** after render.

Q: Can I use my GPU? A: Yes. Choose an encode mode in Settings → Rendering or in the render dialog when available.

Q: Where is the full documentation? A: Use **Help** → **Documentation** in the desktop app.

Q: Does the Flyover Map widget need an API key? A: Not by default. The built-in OpenStreetMap tile template works without an API key, and you can also point the widget at your own raster tile source.

Q: My GoPro recording is split into multiple files. Do I need to join them first? A: No. Use **+ Multi** in the Media files tab to load all chapter files. The renderer stitches them automatically — no transcoding, no quality loss.

Q: Can I use custom icons? A: Yes. Place a normalized SVG in `public/icons/{widgetType}/` inside the app's resources folder and it will appear in the icon picker automatically.

Q: Can I import activities from Strava? A: Yes. Configure your Strava OAuth credentials in **Settings** → **Integrations** (desktop) or in `.env.local` (web), then click **Strava** in the editor header. Activities are

downloaded as TCX files and include GPS, altitude, heart rate, cadence, power, and temperature where recorded. You no longer need a separate FIT file for HR or power when importing from Strava.

Q: Can I import activities from Garmin Connect? A: Yes. Create a Personal API key on the Garmin Connect Personal API page, paste it in **Settings** → **Integrations** → **Garmin Connect**, then click **Garmin** in the editor header. The app downloads the original FIT file your device recorded. See Importing from Garmin Connect.

Thank you for using VeloOverlay Toolkit.